

Decisiones Arquitectónicas (ADRs)

Entregable 4 – Práctica de Arquitectura y Diseño de Sistemas 2026

ID	Título	Estado	Fecha
ADR-002	Estrategia de persistencia de información del sistema	Aceptada	01/06/2026

1. Contexto

El sistema debe almacenar información relacionada con usuarios, complejos deportivos, canchas, reservas, equipamiento, pagos e historial de operaciones.

Estas entidades presentan múltiples relaciones entre sí y requieren mantener consistencia en operaciones críticas, especialmente durante la creación y cancelación de reservas.

Motivador 1 – Consistencia de datos

Las reservas no deben generar conflictos de disponibilidad ni inconsistencias en el estado de las canchas.

Motivador 2 – Relaciones complejas

Las funcionalidades principales requieren consultas que involucren múltiples entidades relacionadas.

Motivador 3 – Facilidad de mantenimiento

Se busca una solución ampliamente conocida por el equipo y compatible con las tecnologías seleccionadas.

2. Alternativas consideradas

MongoDB (NoSQL)	Gran flexibilidad de esquema y facilidad para almacenar documentos heterogéneos.	Las relaciones complejas y la necesidad de transacciones dificultan la implementación de reglas de negocio críticas y reportes.
MySQL	Soporta transacciones y es ampliamente conocido.	Posee menor flexibilidad en tipos avanzados y menor experiencia previa del equipo respecto a PostgreSQL.
PostgreSQL (elegida)	Soporta transacciones ACID, relaciones complejas y consultas analíticas de forma nativa.	(esta fue la elegida)

3. Decisión tomada

Se decide: utilizar PostgreSQL como única base de datos relacional para toda la persistencia transaccional del sistema.

Fundamentación

Razón 1: PostgreSQL permite garantizar la consistencia transaccional requerida por operaciones críticas como reservas, pagos y actualización de inventario, satisfaciendo el motivador de consistencia transaccional.

Razón 2: el modelo de datos del sistema es inherentemente relacional y requiere numerosas asociaciones y consultas complejas, lo que responde al motivador de complejidad de las relaciones.

Razón 3: la experiencia previa del equipo con SQL y PostgreSQL reduce el riesgo de errores de implementación y disminuye los tiempos de desarrollo.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Las restricciones de integridad pueden implementarse directamente en la base de datos.	El esquema resulta más rígido y requiere migraciones explícitas ante cambios.
Las operaciones críticas pueden ejecutarse dentro de transacciones ACID.	La escalabilidad horizontal es más compleja que en soluciones NoSQL.
Las consultas estadísticas y reportes pueden implementarse eficientemente mediante SQL.	Cambios importantes en el modelo pueden requerir refactorizaciones del esquema.